

PGMR — Sistema de Rádio

Músicas Faltando

Documentação Técnica Consolidada

Gerado em: 24/03/2026 18:07:51

1. Visão Geral

O módulo "Músicas Faltando" é responsável por detectar músicas capturadas nas rádios monitoradas que não existem no acervo local da emissora, e providenciar o download automático via Deezer/Deemix.

Fluxo Principal:

- Scraping captura músicas das rádios → scraped_songs (Supabase)
- Grade builder verifica acervo local (findSongMatch com similaridade)
- Música não encontrada → adicionada à lista de faltando (missingSongs)
- Download automático via Deemix com prioridade por urgência
- Após download → ID3 enrichment + roteamento por gênero/pasta

Arquivos do Módulo:

Arquivo	Responsabilidade
MissingView.tsx	UI: lista faltando, download manual/lote, histórico
useGlobalDownloadService.ts	Fila sequencial com mutex, prioridade, retry
useCapturedDownloadService.ts	Poll Supabase + download de capturadas
useCheckMusicLibrary.ts	Verificação de existência com similaridade
downloadMutex.ts	Mutex global com watchdog de 90s
autoDownloadStore.ts	Estado da fila, ARL, stats diários
libraryVerificationCache.ts	Cache de verificação com TTL de 3min
radioStore.ts (MissingSong)	Modelo de dados e actions

2. Modelo de Dados

2.1 MissingSong (radioStore.ts)

```
interface MissingSong {
  id: string;
  title: string;
  artist: string;
  station: string;           // Emissora de origem
  timestamp: Date;
  status: 'missing' | 'downloading' | 'downloaded' | 'error';
  dna?: string;              // DNA/fingerprint da música
  urgency?: 'grade' | 'sequence' | 'normal';
}
```

Níveis de Urgência:

- grade (+500 prioridade) — música necessária para bloco da grade em construção
- sequence (+200 prioridade) — música na sequência de programação
- normal (0) — captura genérica de monitoramento

2.2 DownloadHistoryEntry

```
interface DownloadHistoryEntry {
  id: string;
  songId: string;
  title: string;
  artist: string;
  timestamp: Date;
  status: 'success' | 'error';
  errorMessage?: string;
  duration?: number;         // tempo de download em ms
}
```

3. Mutex Global de Downloads

3.1 downloadMutex.ts

Garante que apenas UM download acontece por vez em todo o sistema (grade, capturadas, manual). Implementa fila com prioridade.

Mecanismo:

- `acquireDownloadLock(priority)` — adquire o lock ou entra na fila ordenada por prioridade
- `releaseDownloadLock()` — libera e passa para o próximo da fila
- Watchdog de 90 segundos — se o lock não for liberado, força release automático

```
const LOCK_SAFETY_TIMEOUT_MS = 90_000;
```

```
acquireDownloadLock(priority = 0) → Promise<void>  
  Se lock livre → trava + inicia watchdog  
  Se lock ocupado → entra na fila (ordenada por priority desc)
```

```
releaseDownloadLock()  
  Cancela watchdog atual  
  Se fila não vazia → passa lock para próximo + reinicia watchdog  
  Se fila vazia → destrava
```

4. Serviço Global de Downloads

4.1 useGlobalDownloadService.ts

Serviço principal que gerencia a fila de downloads automáticos. Roda em background independente da view.

Ciclo de Vida:

- start() — inicia polling a cada 100s + check ARL a cada 15min
- checkNewMissingSongs() — detecta novas músicas faltando no store
- processQueue() — processa fila sequencialmente com mutex

Priorização da Fila:

```
PRIORITY_GRADE_BOOST    = 500  // Música da grade (máxima prioridade)
PRIORITY_SEQUENCE_BOOST = 200  // Música da sequência
PRIORITY_STATION_BOOST  = 100  // Estação com prioritizeDownloads
Ranking bonus           = 50-1 // Posição no ranking TOP25
```

Retry com Fallback de Qualidade:

- 1ª tentativa: qualidade configurada (ex: MP3_320)
- Se falhar: tenta MP3_128 como fallback
- Se falhar de novo: vai para o fim da fila
- Após 3 falhas totais: removida permanentemente

Filtros de Segurança:

- isVinhetaOrJingle() — bloqueia vinhetas/jingles
- isStationAllowedForDownload() — só baixa de estações da sequência
- blockedSongsEngine — verifica lista de bloqueio
- aliasEngine — aplica correções de nome antes do download
- ARL health check — pausa fila se ARL inválida

Pós-Download:

- markSongAsDownloaded() — atualiza cache de verificação
- readId3Genre() — lê gênero do ID3 tag
- Atualiza scraped_songs no Supabase (ai_genre, ai_energy, year)
- routeFileByGenre() — move arquivo para subpasta de gênero
- Station folder routing — move para pasta da estação se configurado

5. Serviço de Capturadas

5.1 useCapturedDownloadService.ts

Serviço complementar que faz polling no Supabase (scraped_songs) a cada 2 minutos para baixar músicas capturadas nas últimas 24h.

Diferenças do Global:

- Fonte: Supabase (scraped_songs) vs radioStore (missingSongs)
- Prioridade: 0 (baixa) — grade tem preferência via mutex
- Deduplica por artist+title antes de enfileirar
- Filtra estações não permitidas e músicas bloqueadas
- ARL check com cache de 10 minutos

Fluxo:

1. Poll Supabase → scraped_songs últimas 24h (limit 500)
2. Deduplica + filtra bloqueadas + filtra estações
3. Para cada música:
 - a. acquireDownloadLock(0) // prioridade mínima
 - b. checkSongInLibrary() → se existe, pula
 - c. downloadFromDeezer()
 - d. releaseDownloadLock()
 - e. Delay 15s entre downloads
4. Retry: até 3 tentativas, falhas vão p/ fim da fila

6. Verificação de Biblioteca

6.1 useCheckMusicLibrary.ts

Hook que verifica se uma música existe no acervo local usando matching por similaridade (Levenshtein distance).

Estratégias de Match:

- findSongMatch (preferido) — threshold configurável (padrão 0.75)
- checkSongExists (fallback) — match exato
- Normalização: remove (Ao Vivo), (Live), (Acústico), [Remix], feat., ft.
- Tenta primeiro com nome normalizado, depois com original

Normalização de Título:

```
normalizeTitle('Song (Ao Vivo)') → 'Song'  
normalizeTitle('Song [Radio Edit]') → 'Song'  
normalizeTitle('Song (feat. Artist)') → 'Song'
```

6.2 libraryVerificationCache.ts

Cache em memória + localStorage para evitar verificações redundantes.

- TTL: 3 minutos
- Máximo: 500 entradas (LRU cleanup)
- Persiste em localStorage com debounce de 2s
- markSongAsDownloaded() atualiza cache imediatamente

7. Interface — MissingView.tsx

View com ~1900 linhas que apresenta a gestão completa de músicas faltando.

Abas:

- Faltando — lista agrupada por estação, busca, download individual/lote
- Histórico — log de downloads com status, duração, retry
- Deezer — status do deemix, versão, teste de busca, instalação

Funcionalidades:

- Download individual via Deezer com feedback visual
- Download em lote com progresso (barra + contador)
- Modo simulação para testes sem Electron/Deemix
- Exportação da lista (TXT agrupado ou CSV)
- Busca externa (Deezer, Spotify, YouTube, Tidal)
- Retry de downloads com falha (individual ou em lote)
- Detecção automática de Python e Deemix
- Instalação assistida do Deemix via pip

Status por Música:

Status	Ícone	Significado
missing	Download ↓	Aguardando download
downloading	Spinner 🔄	Download em andamento
downloaded	Check ✓ (verde)	Baixado com sucesso
error	X (vermelho)	Falha no download

8. AutoDownload Store

8.1 autoDownloadStore.ts

Store Zustand dedicado ao estado do serviço de download automático.

Estado:

- queueLength — tamanho atual da fila
- isProcessing — se está processando downloads
- activeDownload — { artist, title, startedAt } do download atual
- arlValid — se a ARL do Deezer está válida
- dailyStats — { downloaded, failed, skipped } do dia
- vozBrasilFailed/Downloading/Progress — estado do Voz do Brasil
- resetCounter — incrementado no daily reset para limpar fila

Integração com Daily Reset:

resetQueue() → zera queueLength, isProcessing, activeDownload
+ incrementa resetCounter (observado pelo download service)
→ download service limpa fila interna + processedSongs

9. Diagrama de Fluxo

